

Heap Sort

堆排序

本页面将简要介绍堆排序。

定义

堆排序（英语：Heapsort）是指利用 [二叉堆](#) 这种数据结构所设计的一种排序算法。堆排序的适用数据结构为数组。

过程

堆排序的本质是建立在堆上的选择排序。

排序

首先建立大顶堆，然后将堆顶的元素取出，作为最大值，与数组尾部的元素交换，并维持残余堆的性质；

之后将堆顶的元素取出，作为次大值，与数组倒数第二位元素交换，并维持残余堆的性质；

以此类推，在第 i 次操作后，整个数组就完成了排序。

在数组上建立二叉堆

从根节点开始，依次将每一层的节点排列在数组里。

于是有数组中下标为 i 的节点，对应的父结点、左子结点和右子结点如下：

```
1 iParent(i) = (i - 1) / 2;  
2 iLeftChild(i) = 2 * i + 1;  
3 iRightChild(i) = 2 * i + 2;
```

性质

稳定性

同选择排序一样，由于其中交换位置的操作，所以是不稳定的排序算法。

时间复杂度

堆排序的最优时间复杂度、平均时间复杂度、最坏时间复杂度均为 $O(n \log n)$ 。

空间复杂度

由于可以在输入数组上建立堆，所以这是一个原地算法。

实现



C++Python

```

1 void sift_down(int arr[], int start, int end) {
2     // 计算父结点和子结点的下标
3     int parent = start;
4     int child = parent * 2 + 1;
5     while (child <= end) { // 子结点下标在范围内才做比较
6         // 先比较两个子结点大小, 选择最大的
7         if (child + 1 <= end && arr[child] < arr[child + 1]) child++;
8         // 如果父结点比子结点的值大, 代表调整完毕, 直接跳出函数
9         if (arr[parent] >= arr[child])
10            return;
11        else { // 否则交换父子内容, 子结点再和孙结点比较
12            swap(arr[parent], arr[child]);
13            parent = child;
14            child = parent * 2 + 1;
15        }
16    }
17 }
18
19 void heap_sort(int arr[], int len) {
20     // 从最后一个节点的父节点开始 sift down 以完成堆化 (heapify)
21     for (int i = (len - 1 - 1) / 2; i >= 0; i--) sift_down(arr, i, len - 1);
22     // 先将第一个元素和已经排好的元素前一位做交换, 再重新调整 (刚调整的元素之前的元素), 直到排序完毕
23     for (int i = len - 1; i > 0; i--) {
24         swap(arr[0], arr[i]);
25         sift_down(arr, 0, i - 1);
26     }
27 }

```

```

1 def sift_down(arr, start, end):
2     # 计算父结点和子结点的下标
3     parent = int(start)
4     child = int(parent * 2 + 1)
5     while child <= end: # 子结点下标在范围内才做比较
6         # 先比较两个子结点大小, 选择最大的
7         if child + 1 <= end and arr[child] < arr[child + 1]:
8             child += 1
9         # 如果父结点比子结点的值大, 代表调整完毕, 直接跳出函数
10        if arr[parent] >= arr[child]:
11            return
12        else: # 否则交换父子内容, 子结点再和孙结点比较
13            arr[parent], arr[child] = arr[child], arr[parent]
14            parent = child
15            child = int(parent * 2 + 1)
16
17
18 def heap_sort(arr, len):
19     # 从最后一个节点的父节点开始 sift down 以完成堆化 (heapify)
20     i = (len - 1 - 1) / 2
21     while i >= 0:
22         sift_down(arr, i, len - 1)
23         i -= 1
24     # 先将第一个元素和已经排好的元素前一位做交换, 再重新调整 (刚调整的元素之前的元素), 直到排序完毕
25     i = len - 1
26     while i > 0:
27         arr[0], arr[i] = arr[i], arr[0]
28         sift_down(arr, 0, i - 1)
29         i -= 1

```

外部链接

- [堆排序 - 维基百科，自由的百科全书](#)

本页面最近更新：2024/5/8 20:33:33, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [NachtgeistW](#), [CamberLoid](#), [Enter-tainer](#), [iamtwz](#), [lr1d](#), [ksyx](#), [Menci](#), [minghu6](#), [ouuan](#), [partychicken](#), [shawlleyw](#), [sshwy](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用